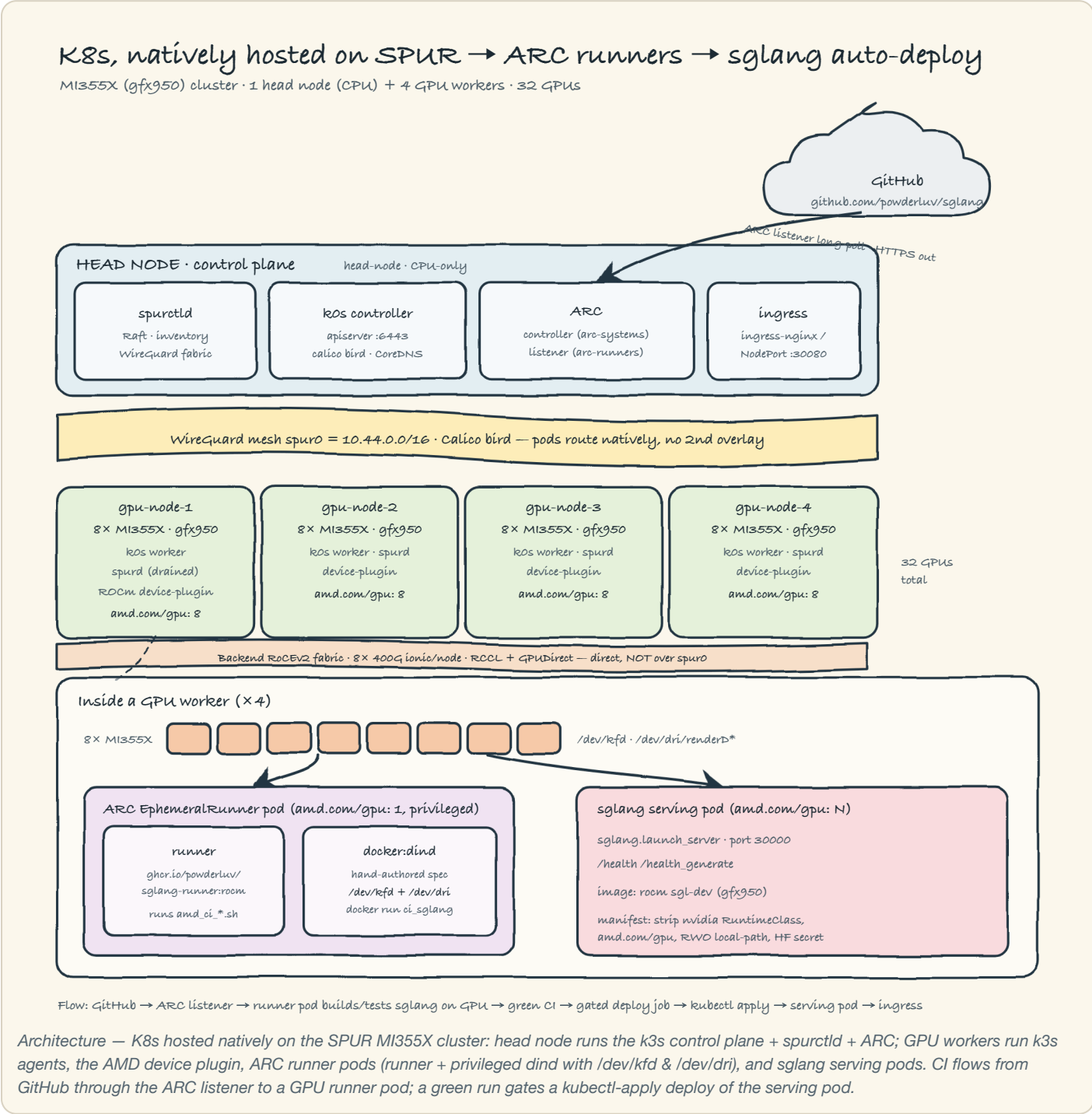


Plan: Native Kubernetes on SPUR + ARC + powderluv/sglang auto-deploy

Status: proposal for review. Target repos: `rocm/spur` (SPUR moved here from `powderluv/spur`), `powderluv/sglang` (fork of `sgl-project/sglang`). Concrete k8s manifests: `spur-examples/gpu-k8s-arc-sglang/`. Author aid: research + 3-way architecture panel + 3 adversarial review passes (feasibility / external-correctness / completeness). Review fixes are folded into the milestones below.



1. Goal & scope

Stand up a **real Kubernetes cluster hosted natively on a SPUR-provisioned cluster**: the SPUR head node becomes the k8s control plane, the SPUR GPU worker nodes become AMD-GPU kubelet nodes, all riding SPUR's existing WireGuard mesh (`spur0`) as the node network. On top of that cluster:

- 1. Install **ARC** (`actions-runner-controller` , the `gha-runner-scale-set` model).
- 2. Fork `sgl-project/sglang` → `github.com/powderluv/sglang` .
- 3. Register a self-hosted **GPU runner scale set** for the fork.
- 4. Run a trimmed AMD CI suite on it, and **auto-deploy an sglang serving Deployment to the same cluster when CI is green**.

The end-state design is **embedded-k0s-as-a-Spur-service** (SPUR owns the k8s lifecycle via `spur k8s up/down` , k0s supervised by a spur-owned unit, GPUs eventually exposed through SPUR's own `spur-devices` CDI). It is delivered **coexist-first**: an early `systemd/k0sctl` bring-up gets ARC + sglang green on the real hardware fast, and nothing above the "who supervises k0s" boundary is rebuilt when k0s later moves under SPUR.

This **inverts** SPUR's only current k8s story (scheduling *onto* an existing cluster) into *hosting* a cluster.

Confirmed target hardware (verified over SSH, 2026-07-09)

Role	Host	Underlay IP	Specs
Head / control plane	<code>head-node</code>	(uplink NIC)	32 vCPU, 125 GB, Ubuntu, CPU-only
GPU worker	<code>gpu-node-1</code>		236 vCPU, 2.7 TB, 8× MI355X (gfx950)
GPU worker	<code>gpu-node-2</code>		same
GPU worker	<code>gpu-node-3</code>		same
GPU worker	<code>gpu-node-4</code>		same

All four workers are **8× AMD Instinct MI355X, gfx950, SPX compute partition** (whole-GPU → a clean `amd.com/gpu: 8` per node; **32 GPUs total**), `/dev/kfd` present, Ubuntu 24.04 / kernel 6.8. **All M0 hard gates already pass**: (1) gfx950 is a sglang-supported arch; (2) nodes are mutually reachable on a flat, routed private /16 underlay (`gpu-node-1`→`gpu-node-4` = 0% loss, 0.3 ms) so a full WireGuard mesh is trivially feasible — no relay overlay needed; (3) every node has direct internet egress (`ghcr.io` reachable). Nothing is installed yet (greenfield: no spur, no k8s). **Access**: `ssh head-node` (head) and `ssh gpu-node-{1..4}` (workers), user the provided login (via the site jump host).

Because the underlay is already flat and sub-millisecond, the WireGuard mesh here is about **encryption + SPUR-native fabric ownership**, not reachability — the "worker↔worker broken by hub-and-spoke" risk that dominates the generic plan is largely moot on this cluster (full-mesh reconciliation is straightforward since every endpoint is directly reachable). k0s can even run on the raw underlay IPs if the mesh is deferred.

2. Current state vs target

Concern	SPUR today (verified in-repo)	Target
K8s relationship	<code>spur-k8s</code> operator is a tenant : a "virtual agent" (<code>agent.rs</code> implements <code>SlurmAgentServer</code>) that registers to <code>spurctld</code> , turns <code>SpurJob</code> CRs into Pods, and imports labeled k8s Nodes into SPUR (<code>node_watcher.rs</code>)	SPUR hosts the cluster: head = control plane, GPU workers = kubelets
Cluster bring-up	None. Only <code>deploy/bare-metal/k8s_test.sh</code> spins a throwaway kind (Docker-in-Docker) cluster with busybox , non-GPU jobs to unit-test the operator	k0s controller on head + k0s workers on GPU nodes, real GPU scheduling
Node network	WireGuard <code>spur0</code> , <code>10.44.0.0/16</code> , UDP 51820, hub-and-spoke (workers peer only the controller; <code>spur net add-peer</code> is manual; <code>spurd</code> registers an empty wg pubkey so nothing auto-reconciles), <code>wg_enabled=false</code> by default, no IP-forwarding , no MTU set	<code>spur0</code> as k8s node-IP network; worker↔worker reachable; MTU pinned
CNI	None (<code>spur-net</code> = a <code>wg / wg-quick</code> shell wrapper + an OCI image puller)	Calico bird (BGP native, no overlay) over <code>spur0</code> ; mesh carries pod CIDRs; pod <code>10.42/16</code> , svc <code>10.43/16</code>
GPU → pods	<code>spur-devices</code> discovers KFD GPUs and emits <code>amd.com/gpu</code> CDI specs + injects <code>/dev/kfd</code> + <code>/dev/dri</code> for spurd jobs only	<code>amd.com/gpu</code> advertised to kubelet (ROCm device plugin first, native spur plugin later)
Long-running services	None. <code>spurd</code> is one-shot (monitor loop waits for exit; time-limit watchdog SIGKILLs). "Service jobs" are roadmap 13.1, unbuilt	A <code>spurd</code> -owned, restart-on-failure, health-checked service unit for k0s
Node roles	No control-plane/worker flag	<code>spur.amd.com/control-plane</code> label + inventory role flag
ARC / runners	None	ARC controller + GPU dind scale set for <code>powderluv/sglang</code>

3. Architecture

Chosen: embedded-k0s-service, delivered coexist-first. k0s does the genuinely hard parts (unmodified upstream `apiserver/scheduler/controller-manager`, real `etcd`, `kube-proxy`, CDI-capable `containerd`, `calico`) supervised from a single binary, bootstrapped declaratively by `k0sctl`. SPUR provides the node network, node inventory, GPU discovery, and eventually lifecycle supervision.

GitHub (github.com/powderluv/sglang)
▲ outbound HTTPS only (ARC listener long-poll)



How GPUs reach pods. Hosts already run the `amdgpu` kernel driver + `ROCm` and expose `/dev/kfd` (shared KFD compute) and `/dev/dri/renderD*` (per-GPU render nodes) — SPUR's own GRES discovery needs exactly these. Phase 1 uses the off-the-shelf **ROCm/k8s-device-plugin** DaemonSet (+ its `node-labeller`) to advertise `amd.com/gpu`; a pod requesting `resources.limits.amd.com/gpu: N` gets the device nodes injected. Phase 3 replaces it with a **native spur-device-plugin** whose `ListAndWatch` is `DeviceRegistry.list()` filtered to GPUs and whose `Allocate` returns the CDI device / device-node mounts built from `spur-devices`, making SPUR the single GPU allocation authority.

How the mesh is used. Every node already has a stable, encrypted `10.44.0.x` address on `spur0`. That is the k8s node IP (`--node-ip`), the apiserver advertise address, and the flannel interface (`--flannel-iface spur0`). Control-plane↔worker is spoke↔hub and works directly. Worker↔worker (cross-node pod traffic, VXLAN, kube-proxy east-west) requires a **full WireGuard mesh** — and full mesh is only possible if workers can reach each other on the underlay (see M0 gate).

Backend RoCEv2 fabric (distributed inference). Separate from `spur0`: each GPU worker also has 8× AMD Pensando (`ionic`) 400 GbE RoCEv2 NICs (one per MI355X, rail-optimized). Multi-node DI (RCCL collectives, Mooncake/MoRI KV-transfer) runs RDMA **directly over this fabric**, attached to pods via `hostNetwork` or Multus+SR-IOV — **never tunneled through the mesh**. `spur0` carries only control plane, Calico pod/service traffic, ARC, and the TCP rendezvous. See M6.5 + `50-rdma/`.

4. Gap analysis

Gap	Where	Size	What to build
Worker↔worker underlay reachability (WireGuard has no relay/DERP)	hardware/topology	—	M0 hard gate ; if absent, adopt a relay overlay (Tailscale/Netbird) or restrict k8s to mutually-reachable nodes
Pod→internet egress (ARC long-poll + multi-GB image pulls)	head iptables / worker routes	S	SNAT/MASQUERADE <code>10.42.0.0/16</code> out head uplink; verify worker egress + CoreDNS→upstream
No cluster bring-up on SPUR nodes	new <code>spur cluster</code> CLI + host	L	Template the k0sctl inventory + k0s systemd units from controller inventory; SSH fan-out from head
Hub-and-spoke can't do worker↔worker; mesh carries no pod CIDRs	<code>spur-net</code> / <code>spurd</code>	M	Fix <code>spurd</code> empty <code>wg_pubkey</code> (<code>reporter.rs</code>); controller fan-out <code>add-peer</code> → full mesh with each node's pod /24 in AllowedIPs (required for Calico <code>bird</code> native routing)
<code>spur0</code> MTU unset → CNI blackhole	<code>spur-net/src/wireguard.rs</code>	S	Pin MTU derived from underlay NIC ; let flannel auto-derive (iface - 50)
No CNI	k0s built-in calico	S	Calico <code>bird</code> (BGP, no overlay) pinned to <code>spur0</code> via <code>ipAutodetectionMethod=interface=spur0</code> ; VXLAN = fallback
Backend RoCEv2 fabric not exposed to pods (DI)	k8s (Multus + SR-IOV/RDMA device plugin) + host	L	Expose 8x <code>ionic</code> RoCE NICs + GPUDirect to pods off <code>spur0</code> ; verify with sglang DI tests (<code>spur-examples/gpu-k8s-arc-sglang/50-rdma/</code>)
GPU device plugin	k8s (ROCm plugin) → later <code>spur-devices</code>	S→L	P1 apply plugin + node-labeller + -health variant ; P3 native plugin
Host firewall blocks intra-mesh k8s ports	host	S	Allow 6443/tcp, 8472/udp (VXLAN), 10250/tcp (+2379-2380 HA) on <code>spur0</code>
Double GPU allocation (<code>spurd</code> + kubelet on same <code>/dev/dri</code>)	<code>spurctld</code> scheduler	M	Enforce in M2, not deferred : DRAIN k8s GPU nodes from SPUR scheduler; P3 single authority
No long-running/service job type	<code>spurd</code>	L	P2 <code>spurd-owned</code> k0s unit (<code>Type=notify</code> , <code>Delegate=yes</code> , <code>KillMode=mixed</code>), not a forked child
No node-role primitive	<code>spur-core</code> config/inventory	S	<code>spur.amd.com/control-plane</code> label + role flag + drain hook
ARC install + GitHub auth	k8s / ARC	M	Two Helm OCI charts; GitHub App secret
GPU runner image (stock has no ROCm)	new GHCR image	M	FROM <code>ghcr.io/actions/actions-runner</code> + ROCm + docker CLI; push <code>ghcr.io/powderluv/sglang-runner</code>
dind GPU passthrough (chart can't customize injected sidecar)	ARC values	M	Unset <code>containerMode</code> ; hand-author the full pod spec with <code>/dev/kfd</code> + <code>/dev/dri</code> in the <code>docker:dind</code> container
Per-runner GPU isolation (<code>/etc/podinfo/gha-render-devices</code>)	ARC pod spec	M	Downward-API file pinning allocated <code>renderD*</code> ; else script sees ALL GPUs
Persistent image cache / registry	GPU nodes / GHCR	M	hostPath docker data-root (or node-local registry); DOCKERHUB creds; GHCR pull secret
sglang fork runner registration	fork workflows	M	Disable all upstream <code>*amd*</code> / <code>nightly*</code> ; add one trimmed workflow whose <code>runs-on</code> matches the scale set
Scale-set name must match arch regex	fork workflows + ARC	S	Name <code>linux-mi35x-gpu-1</code> / <code>linux-mi35x-gpu-1</code> (matches <code>^linux-(mi[0-9]+[a-z]*)-gpu-[0-9]+</code>)
LOCAL_DOCKER_REGISTRY hardcode <code>10.44.14.109:5000</code> (inside mesh CIDR)	fork script	S	Pass <code>--custom-image</code> to bypass, or patch script; exclude that IP from mesh AddressPool

Gap	Where	Size	What to build
AMD sglang serving manifest	sglang fork	S	New manifest: strip nvidia RuntimeClass, <code>amd.com/gpu</code> , RWO local-path/emptyDir, securityContext, HF secret
Gated auto-deploy job	fork workflows	M	Non-GPU deploy scale set + kubectl + dedicated SA/RBAC + in-cluster token
External access to served model	k8s	S	Install ingress-nginx (NodePort) over mesh — k0s bundles none; DNS if public
Observability	k8s + <code>spur-metrics</code>	M	metrics-server, GPU exporter, sglang <code>/metrics</code> , ARC listener metrics, logs
Teardown ordering	ops	S	Uninstall runner sets before controller; <code>k0sctl reset</code> ; reverse SPUR drain; reconcile mesh
<code>[cluster]</code> config section	<code>spur-core/src/config.rs</code>	S	New <code>[cluster]</code> + <code>[cluster.arc]</code> , distinct from the inverse-direction <code>[kubernetes]</code>
Native lifecycle RPCs	<code>spur-proto</code> + <code>spurtld</code> + <code>spurd</code>	L	P2 <code>ClusterUp/Down/Status</code> , <code>StartClusterComponent</code> ; spur k8s CLI

5. Milestone plan

Milestones are ordered so the highest silent-failure-risk items are **gates** before anything is built on top.

Concrete artifacts are the source of truth, not the snippets below. The k0s cluster, GPU device plugin, ARC controller + runner scale sets, and sglang serving + fork CI/CD are authored as real files in `spur-examples/gpu-k8s-arc-sglang/` (`10-cluster/k0sctl.yaml`, `20-gpu/`, `30-arc/`, `40-serving/`, `bootstrap.sh` / `teardown.sh`, `RUNB00K.md`, `versions.lock.md`). Milestone snippets here are illustrative; run the tree.

M0 — Preflight gates + network prerequisites

Hard gates — all three verified PASSING on this cluster (2026-07-09); re-check only if hardware changes:

- GPU arch gate —  gfx950 (MI355X).** `rocm-smi` on `gpu-node-1` and `gpu-node-4` report 8x *AMD Instinct MI355X*, `gfx950`, SPX partition. `gfx950` is a sglang-supported arch → use the `mi35x` CI/serving image path and the `linux-mi35x-gpu-1` scale-set name. (No descope/custom-build needed; this is datacenter MI355X, not the workspace's usual `gfx1201`.)
- Worker↔worker underlay gate —  directly reachable.** All nodes sit on a flat routed private /16; `gpu-node-1` → `gpu-node-4` is 0% loss / 0.3 ms. Full WireGuard mesh is trivial; no Tailscale/Netbird relay needed. (Confirm UDP/51820 specifically once WireGuard is up in step 4.)
- Egress gate —  direct per-node internet.** Head and workers each reach `https://ghcr.io/v2/` (401 = reachable, no proxy). Pod-CIDR SNAT is still wired below for pod egress, but no head-funnel is required.

Setup: 4. Mesh: on head (`head-node`, real IP `<head-ip>`) `sudo spur net init --cidr 10.44.0.0/16 --port 51820` → head gets `spur0 = 10.44.0.1`; per worker `gpu-node-N` `sudo spur net join --endpoint <head-ip>:51820 --server-key <pub> --address 10.44.0.<2..5>` then on head `sudo spur net add-peer --key <node-pub> --allowed-ip 10.44.0.<2..5>/32 --endpoint <node-underlay-ip>:51820` (underlay IPs: `gpu-node-1=`, `gpu-node-2=`, `gpu-node-3=`, `gpu-node-4=`). Suggested mesh map: head 10.44.0.1, workers `.2/.3/.4/.5`. 5. **Full mesh** (preferred) via §6 reconciliation, or **stopgap** `sudo sysctl -w net.ipv4.ip_forward=1` on the head (persist in `/etc/sysctl.d/99-spur.conf`) for first light only — funnels all cross-node/GPU-collective traffic through the head. 6. **Pod egress SNAT** on the head: `sudo iptables -t nat -A POSTROUTING -s 10.42.0.0/16 -o <head-uplink> -j MASQUERADE` (persist). Verify a worker-scheduled pod resolves DNS and pulls from `ghcr.io/docker.io` **before** installing ARC. 7. **MTU:** read the underlay NIC MTU, set `spur0` accordingly (wg-quick default ≈ underlay – 80), and let flannel auto-derive (`iface - 50`); only override flannel downward if needed. Do not hardcode 1420 blindly. 8. **Host firewall:** allow on `spur0` — 6443/tcp (apiserver), 10250/tcp (kubelet), **179/tcp (Calico BGP, bird)**, 8132/tcp (k0s connectivity), 9443/tcp (k0s API), 2379-2380/tcp (etcd, HA). (VXLAN fallback also needs 4789/udp.) `firewalld/ufw` block these by default. 9. **CIDR discipline:** pod `10.42.0.0/16`, service `10.43.0.0/16`, both clear of mesh `10.44.0.0/16`. Reserve/exclude `10.44.14.109` from the mesh `AddressPool` (sglang hardcodes it as a registry).

Done when: each gate passes; from worker-1, `ping -c3 10.44.0.3` and a **pod-to-pod cross-node** DF probe at ~1370 payload succeeds (host ping alone does not exercise the CNI/VXLAN MTU path — validate it in M1); `wg show spur0` lists the expected peers; a worker pod pulls a public image.

M1 — k0s bring-up via k0sctl (coexist), nodes Ready

Concrete config: `spur-examples/gpu-k8s-arc-sglang/10-cluster/k0sctl.yaml` (fill real SSH addresses + pin the k0s version). Single controller = head (control-plane only, no kubelet), four workers = GPU nodes, Calico **bird** (BGP native, no overlay) pinned to `spur0`. Pods route directly over the WireGuard mesh — this requires SPUR to put each node's pod /24 in its peer AllowedIPs (see §6); the VXLAN fallback in `k0sctl.yaml` works with stock SPUR meanwhile.

```
# from your workstation (needs the k0sctl binary + SSH to all nodes):
k0sctl apply --config spur-examples/gpu-k8s-arc-sglang/10-cluster/k0sctl.yaml
k0sctl kubeconfig --config spur-examples/gpu-k8s-arc-sglang/10-cluster/k0sctl.yaml > spur-examples/gpu-k8s-arc-sglang/kubeconfig
export KUBECONFIG=$PWD/spur-examples/gpu-k8s-arc-sglang/kubeconfig
```

k0s puts the kubelet root at `/var/lib/k0s/kubelet` (matters for M2). The head is a `controller` role → it runs the control plane as supervised processes (not pods) and schedules no workloads. Label GPU workers `spur.amd.com/compute=true` (`spur-examples/gpu-k8s-arc-sglang/20-gpu/node-labels.sh`).

Addons k0s does **not** bundle (see `spur-examples/gpu-k8s-arc-sglang/15-addons/`): install **local-path-provisioner** (RWO storage; mark default) and **ingress-nginx** (NodePort) — the standard upstream components, pinned in `versions.lock.md`.

Done when: `kubectl get nodes -o wide` shows the 4 workers `Ready` with `INTERNAL-IP = 10.44.0.x` (the controller is a control-plane node, not a schedulable kubelet); a **cross-node** pod-to-pod DF transfer at ~1370 B succeeds (validates calico-over-mesh **and** MTU).

M2 — GPU device plugin + node-labeller + validate a GPU pod + close the double-allocation footgun

Apply `spur-examples/gpu-k8s-arc-sglang/20-gpu/amd-device-plugin.yaml` — the ROCm device plugin + node-labeller, with the **k0s kubelet path** (`/var/lib/k0s/kubelet/device-plugins`, the one k0s-specific delta vs the upstream manifest):

```
kubectl apply -f spur-examples/gpu-k8s-arc-sglang/20-gpu/amd-device-plugin.yaml
bash spur-examples/gpu-k8s-arc-sglang/20-gpu/node-labels.sh # first run: label GPU workers by name (chickadee)
kubectl get nodes -o json | jq '.items[].status.allocatable["amd.com/gpu"]'
```

Validation pod: `rocm/rocm-terminal` running `rocminfo` with `resources.limits.amd.com/gpu: 1` and `nodeSelector: {spur.amd.com/compute: "true"}`.

Close double-allocation now (do not defer to native plugin): DRAIN the k8s-owned GPU nodes from the SPUR scheduler so `spurctld` cannot place a spur job on a GPU that kubelet is using. Options, pick one and record it:

- `scontrol / spur node` set the GPU nodes to a `DRAIN` /down state in `spurctld` (keep `spurd` for mesh/inventory only), or
- stop `spurd`'s agent/scheduler role on those nodes and run the mesh via standalone `wg-quick`, or
- pin disjoint `R0CR_VISIBLE_DEVICES` per manager if you truly must share a node.

Done when: `allocatable["amd.com/gpu"]` is non-zero per worker; `kubectl logs rocminfo` prints the expected `gfx*` agents; the GPU nodes are confirmed un-schedulable by `spurctld` (`spur nodes` shows them drained).

M3 — Fork sglang, disable its AMD/nightly workflows, install ARC + auth

1. Fork → `github.com/powderluv/sglang`.
2. **Disable every upstream AMD/nightly workflow** so nothing queues on your single pool or hard-fails on missing org secrets. Grep and handle all of:

```
ls .github/workflows | grep -Ei 'amd|nightly'
# amd-aiter-scout, amd-ci-job-monitor, nightly-*amd*, pr-test-amd*, release-docker-amd*
```

Rename to `.disabled` or guard with `if: github.repository == 'sgl-project/sglang'`. Note: `pr-test-amd.yml`'s 1-GPU jobs use the label `linux-mi325-1gpu-sglang` and would **execute and contend** on your runners (they don't "hang"); disable them so your GPU pool stays dedicated to the fork CI. Also disable scheduled workflows in the fork's Actions settings.

3. GitHub **App** (recommended over PAT): install on the fork; permissions per the current ARC docs (historically ~ Repository → Actions: Read, Administration: Read & Write, Metadata: Read — validate against live docs at deploy). Record App ID + Installation ID, generate `.pem`.
4. Secret + controller:

```
kubectl create ns arc-runners
kubectl -n arc-runners create secret generic sglang-arc-app \
  --from-literal=github_app_id=<APP_ID> \
  --from-literal=github_app_installation_id=<INSTALL_ID> \
  --from-file=github_app_private_key=./sglang-app.private-key.pem
helm install arc -n arc-systems --create-namespace \
  oci://ghcr.io/actions/actions-runner-controller-charts/gha-runner-scale-set-controller
```

Done when: `arc-systems` controller pod `Running`; `helm list -n arc-systems` shows `arc`; a worker pod still has verified internet egress (M0#6).

M4 — GPU runner image + hand-authored dind scale set (with GPU in the dind sidecar)

Runner image (stock `ghcr.io/actions/actions-runner` has no ROCm or docker CLI):

```
FROM ghcr.io/actions/actions-runner:latest
USER root
RUN apt-get update && apt-get install -y docker.io <rocm runtime pkgs> && \
  usermod -aG docker,video,render runner
USER runner
```

Push `ghcr.io/powderluv/sglang-runner:rocm`. Add a GHCR `imagePullSecret` in `arc-runners`.

Scale set — do NOT use `containerMode: dind`. ARC auto-injects and locks the dind sidecar when `containerMode` is set; you cannot add `/dev/kfd` + `/dev/dri` to it via values (ARC issues #3709/#4416). `sglang's` `amd_ci_start_container.sh` runs `docker run --device=/dev/kfd --device=/dev/dri ...` against the **dind daemon**, so the devices must live in the **dind container**. Unset `containerMode` and hand-author the full pod spec:


```

githubConfigUrl: https://github.com/powderluv/sclang
githubConfigSecret: sclang-arc-app
runnerScaleSetName: linux-mi35x-gpu-1      # MUST match ^linux-(mi[0-9]+[a-z]*)-gpu-[0-9]+ (see M5)
minRunners: 0
maxRunners: 2
# containerMode intentionally OMITTED – full spec below
template:
  spec:
    nodeSelector: {spur.amd.com/compute: "true"}
    imagePullSecrets: [{name: ghcr-pull}]
    initContainers:
      - name: init-dind-externals          # copy runner externals into shared vol (from stock dind template)
        image: ghcr.io/powderluv/sclang-runner:rocm
        command: ["cp", "-r", "/home/runner/externals/.", "/home/runner/tmpDir/"]
        volumeMounts: [{name: dind-externals, mountPath: /home/runner/tmpDir}]
    containers:
      - name: runner
        image: ghcr.io/powderluv/sclang-runner:rocm
        command: ["/home/runner/run.sh"]
        env: [{name: DOCKER_HOST, value: unix:///run/docker/docker.sock}]
        resources: {limits: {amd.com/gpu: 1}}
        securityContext: {privileged: true}
        volumeMounts:
          - {name: dind-sock, mountPath: /run/docker}
          - {name: podinfo, mountPath: /etc/podinfo}      # per-runner GPU pinning (below)
      - name: dind
        image: docker:dind
        securityContext: {privileged: true}
        args: ["dockerd", "--host=unix:///run/docker/docker.sock"]
        volumeMounts:
          - {name: dind-sock, mountPath: /run/docker}
          - {name: dind-externals, mountPath: /home/runner/externals}
          - {name: dev-kfd, mountPath: /dev/kfd}          # devices INTO the dind daemon
          - {name: dev-dri, mountPath: /dev/dri}
    volumes:
      - {name: dind-sock, emptyDir: {}}
      - {name: dind-externals, emptyDir: {}}
      - {name: dev-kfd, hostPath: {path: /dev/kfd, type: CharDevice}}
      - {name: dev-dri, hostPath: {path: /dev/dri, type: Directory}}
      - name: podinfo
        downwardAPI:
          items: [{path: gha-render-devices, fieldRef: {fieldPath: metadata.annotations['spur.amd.com/render-devices']}}]

```

- **Per-runner GPU isolation:** sclang reads `/etc/podinfo/gha-render-devices` to pin specific `renderD*` nodes; without it the script falls back to `--device /dev/dri` (ALL GPUs), re-introducing double-allocation *inside* k8s. Populate it from the device-plugin/CDI allocation (fold into M2/M8). As an interim, run `maxRunners: 1` per node so "all GPUs" == "its GPUs".
- **Image cache:** each EphemeralRunner is cold (`minRunners: 0`) and re-pulls multi-GB `rocm/sgl-dev`. Add a persistent docker data-root (hostPath per GPU node) or a node-local registry; supply `DOCKERHUB_AMD_USERNAME` / `TOKEN` fork secrets to dodge anonymous rate limits.

```

helm install linux-mi35x-gpu-1 -n arc-runners -f sclang-runners-values.yaml \
oci://ghcr.io/actions/actions-runner-controller-charts/gha-runner-scale-set

```

Done when: `arc-runners AutoscalingListener` Running ; the scale set appears under the fork's Settings → Actions → Runner scale sets; a manually-triggered no-op job lands an EphemeralRunner pod with a **running dind container that sees `/dev/kfd`** (`kubectl exec ... -c dind -- ls /dev/kfd`).

M5 — Point the trimmed fork workflow at the scale set

sglang's `amd_ci_*.sh` scripts detect GPU arch from the **pod hostname** via `^linux-(mi[0-9]+[a-z]*)-gpu-[0-9]+`. The ephemeral pod hostname is `<runnerScaleSetName>-...`, so the scale-set name **must** match that regex:

- MI300/MI325 (gfx942): `linux-mi35x-gpu-1` (or rely on the mi30x default explicitly).
- MI35x (gfx950): `linux-mi35x-gpu-1`. `linux-mi325-**1gpu**-sglang` does **not** match and silently defaults to the mi30x image — fatal on gfx950.

Add one workflow `.github/workflows/fork-ci.yml`:

```
name: Fork AMD CI
on: {workflow_dispatch: {}, push: {branches: [main]}}
jobs:
  amd-gpu-test:
    runs-on: linux-mi35x-gpu-1          # == runnerScaleSetName, matches the arch regex
    steps:
      - uses: actions/checkout@v4
      - run: bash scripts/ci/amd/ensure_vram_clear.sh rocm
      - run: bash scripts/ci/amd/amd_ci_start_container.sh --custom-image rocm/sgl-dev:<pinned-tag>
        # --custom-image bypasses the hardcoded LOCAL_DOCKER_REGISTRY=10.44.14.109:5000 probe
      - run: bash scripts/ci/amd/amd_ci_install_dependency.sh
      - run: bash scripts/ci/amd/amd_ci_exec.sh -w /sglang-checkout/test \
        python3 run_suite.py --hw amd --suite stage-a-test-1-gpu-small-amd
```

Add an **arch-assertion** step (`rocminfo | grep gfx942` or `gfx950`) before the suite so a mis-detected image fails loudly.

Provide fork secrets `HF_TOKEN` (+ optional `DOCKERHUB` creds).

Done when: `gh workflow run fork-ci.yml -R powderluv/sglang` spawns an EphemeralRunner on a GPU node, `ci_sglang` launches against the correct ROCm image, the arch assertion passes, and the small suite is green.

M6 — Gated auto-deploy of sglang serving

Author `deploy/k8s/sglang-service-amd.yaml` on the fork. Adapt `docker/k8s-sglang-service.yaml`, and strip the three fatal NVIDIA-isms:

- Delete** `runtimeClassName: nvidia` **and** the `RuntimeClass/nvidia` object (no such handler on AMD/k0s → `RuntimeClassNotFound`).
- `nvidia.com/gpu` → **amd.com/gpu: N**; image → `ROCm (ghcr.io/powderluv/sglang:<tag> or rocm/sgl-dev:<tag>)`.
- PVC:** the upstream 30Gi `ReadWriteMany` `storageClassName: default` never binds on k0s until you install local-path (RWO). Use `ReadWriteOnce` + `storageClassName: local-path`, or `emptyDir` if the HF cache need not persist.
- Service:** upstream `type: LoadBalancer` stays `<pending>` with `servicelb` disabled. Use ingress-nginx via NodePort (see M7-ops); with k0s nothing is bundled to fight — you install ingress-nginx + local-path yourself.
- securityContext:** `supplementalGroups: [<video-gid>, <render-gid>]`, `seccompProfile.type: Unconfined`, and `privileged/ SYS_PTRACE` as ROCm needs. The device plugin injects `/dev/kfd` + `/dev/dri` via `amd.com/gpu`; `--group-add / --device` are not valid Deployment fields.
- HF secret:** create `HF_TOKEN` secret in the serving namespace, reference via `secretKeyRef`; document HF license acceptance, or default the auto-deploy to an **ungated** model. Keep `livenessProbe /health`, `readinessProbe /health_generate`, port 30000.

Deploy job — run it **off** the GPU pool so a `kubectl apply` never consumes/queues a GPU. Either a tiny non-GPU ARC scale set (`linux-cpu-deploy`) with `kubectl` baked in, or `ubuntu-latest` with a kubeconfig secret:

```
deploy:
  needs: [amd-gpu-test]
  if: success()
  runs-on: linux-cpu-deploy
  steps:
    - uses: actions/checkout@v4
    - run: kubectl apply -f deploy/k8s/sglang-service-amd.yaml
```

Wire RBAC: create a dedicated ServiceAccount referenced via `template.spec.serviceAccountName` (`automountServiceAccountToken: true`), a `Role / RoleBinding` for `deployments, services, pods, pvc` in the serving namespace, and have the job use the in-cluster token (`--server https://kubernetes.default ...`), not an unspecified kubeconfig.

Done when: on green CI, `kubectl -n sglang rollout status deploy/sglang` succeeds; a `/generate` request returns tokens (reached per M7-ops).

M6.5 — Backend RoCEv2 RDMA fabric + distributed inference (DI)

Required only for multi-node serving (the single-node ARC + sglang loop above does not need it). Concrete artifacts: `spur-examples/gpu-k8s-arc-sglang/50-rdma/`.

Confirmed hardware (SSH, 2026-07-09): each GPU worker has **8x AMD Pensando (ionic) 400 GbE RoCEv2 NICs** — one per MI355X GPU, rail-optimized (3.2 Tb/s/node), separate from the 200 G `ens3` frontend that carries `spur0`. RoCE **v2**, GPUDirect-capable (`amdgpu dmabuf + ionic_rdma 25.08`), ROCm 7.0.1. Backend netdevs are currently **un-IP'd** (RoCEv2 v2 GIDs need per-rail IPs). This is AMD's documented MI355X + 8x-Pensando reference config for sglang DI (MoRI / Mooncake).

The RDMA data plane must NOT ride spur0 — WireGuard can't carry RDMA and would destroy bandwidth/latency. The RoCE fabric attaches to pods directly; only the TCP rendezvous uses the pod network.

Tasks:

- Fabric prereqs (confirm with provider):** assign RoCEv2 rail IPs to the 8 backend netdevs/node; confirm PFC/DCQCN-ECN on NICs + switches; RDMA netns mode (`shared whole-node / exclusive fractional`).
- Expose to pods** — model A (whole-node `hostNetwork: true` → all 8 NICs + `/dev/infiniband`) or model B (**Multus + SR-IOV device plugin** `isRdma:true` → `amd.com/roce_ionic` + per-rail `NetworkAttachmentDefinition`s). Calico stays primary CNI.
- GPUDirect + RCCL:** serving image ships RCCL + RDMA transport; `NCCL_IB_DISABLE=0`, `NCCL_IB_HCA=ionic_0..7`, `NCCL_IB_GID_INDEX=3`, `NCCL_SOCKET_IFNAME=ens3` (rendezvous only).
- Verify with sglang's own DI tests** (`50-rdma/di-verify.md`): reuse `nightly-amd-mi355x-disagg.yml` (2-node 1P1D disaggregation, runner label `linux-sglang-mi35x-di`). Its DI CI is **Slurm-driven** (`salloc 2 nodes` → `scripts/ci/slurm/launch_mi355x.sh`); since **SPUR is Slurm-compatible**, point that `salloc` at **SPUR** (SPUR schedules the 2 MI355X nodes on the RoCE fabric) — Path 1 — or run the benchmark client against a k8s-deployed 2-node sglang — Path 2. (M2's SPUR drain becomes a partition/time split, not a permanent drain, so SPUR can reclaim nodes for a DI run.)

Done when: `ibv_devinfo` in a pod/job lists `ionic_0..7` `PORT_ACTIVE` (container RDMA ABI matches host — see risk); cross-node `ib_write_bw` ~line rate; sglang's 2N 1P1D disagg benchmark completes in range.

M7-ops — External access, observability, teardown

- External access:** use ingress-nginx (installed in M1) with an Ingress for the sglang Service, or expose a `NodePort` and document reaching it over the mesh; add DNS/TLS if public exposure is in scope. `spur0` is private, so even ClusterIP is only reachable from mesh nodes today.
- Observability:** metrics-server; a GPU exporter (ROCm `device-metrics-exporter`); scrape sglang `/metrics`; ARC listener metrics; centralized logs. Wire the existing `spur-metrics` crate to surface cluster/runner health.
- Teardown (spur k8s down) with correct ordering:** `helm uninstall` the **runner scale sets before** the controller (AutoscalingRunnerSet finalizers hang otherwise) → delete namespaces → `k0sctl reset` (or `k0s reset` per node) → **reverse the SPUR scheduler drain** (re-admit GPU nodes) → remove `/etc/cdi` specs + node labels → reconcile the WireGuard mesh back to hub-and-spoke.

M8 — Native integration: move the SAME k0s under SPUR

- `crates/spurd/src/cluster.rs`: SPUR **owns a systemd/OpenRC unit** for k0s (`Type=notify`, `Delegate=yes`, `KillMode=mixed`) and reconciles/health-checks it — it does **not** fork k0s as a child (containerd/kubelet would orphan and leak mounts/cgroups on spurd restart). This is the concrete down-payment on roadmap 13.1 "Service Jobs," scoped as "spurd supervises a unit," not "spurd is PID1."
- `spurctld ClusterController` (new `spur-cluster` crate or `cluster_k8s.rs`): pick control-plane vs agent from inventory, mint token, allocate node IPs via `spur-net::address::AddressPool`, pick non-overlapping CIDRs, template the `k0sctl/k0s` config, drive up/down/reconcile.
- `spur-proto`: `ClusterUp/ClusterDown/ClusterStatus` (controller) + `StartClusterComponent/StopClusterComponent/GetClusterComponentStatus` (agent), on the existing tonic wiring.

4. `spur-cli/src/k8s.rs` : `spur k8s up|down|status|kubeconfig|join|addon` .
5. On join, wire `spur-devices::cdi::discovery::discover_to_cdi()` to write `/etc/cdi` (spurd already depends on `spur-devices`).

Done when: `spur k8s up` reproduces the M1–M2 cluster with no manual `k0sctl apply` ; killing k0s is auto-recovered by the spurd-owned unit; ARC + sglang (M4–M6) run unchanged.

M9 — Full fidelity + HA

1. Native **spur-device-plugin** (Rust/tonic) implementing the kubelet Device Plugin API on `/var/lib/k0s/kubelet/device-plugins/` : `ListAndWatch` ← `DeviceRegistry.list()` GPUs; `Allocate` ← `build_job_injection_plans / a CDIDevice{amd.com/gpu=<id>}` . Use a **stable device ID** (CDI device name / `unique_id`), and drop the dense `0..N ROCR_VISIBLE_DEVICES` mask (wrong when a pod sees only a subset of `/dev/dri` ; AMD's own plugin sets no such env). SPUR becomes the single GPU authority → double-allocation gone.
2. RKE2 / embedded-etcd for an HA control plane.
3. Node join/leave auto-updates **both** the WireGuard full mesh **and** k0s membership.

6. SPUR code changes (by phase)

Phase 0/1 (bring-up):

- `crates/spurd/src/reporter.rs` : stop sending an empty `wg_pubkey` ; carry the node's real WireGuard pubkey so the controller can reconcile peers.
- `crates/spur-net/src/wireguard.rs` : (a) pin MTU on the interface (derived from underlay NIC); (b) controller fan-out `add_peer` so each worker peers every other worker (full mesh), replacing hub-and-spoke, **with each peer's pod /24 added to AllowedIPs** (+ a route to `spur0`) so Calico `bird` native routing works — the mesh becomes the pod network instead of a tunnel under a VXLAN overlay. Needs deterministic per-node pod CIDRs (`--allocate-node-cidrs`).
- `crates/spur-cli/src/net.rs` : `spur net mesh reconcile` (or fold into `add-peer`) driving the full-mesh fan-out from inventory.
- **New** `crates/spur-cli/src/cluster.rs` (registered in `main.rs` , same flat pattern as `net / node`): `spur cluster init|join|gpu-setup|arc|status|destroy` ; Phase-1 templates the k0sctl inventory + k0s units and SSH-fans-out from the head, reading inventory over REST (6820).
- `crates/spur-core/src/config.rs` : `[cluster]` (`distro` , `pod_cidr` , `service_cidr` , `cni_mtu` , `flannel_iface` , `control_plane_node` , `gpu_worker_selector` , `device_plugin`) + `[cluster.arc]` (`installation_name` , `github_config_url` , `container_mode` , `runner_image`), **distinct** from the existing inverse-direction `[kubernetes]` .
- Reuse `deploy/k8s/operator.yaml` , `Dockerfile` , `install.sh` from `users/powderluv/ci-dispatch` where applicable; ARC values/manifests live alongside.
- Scheduler **drain hook** for GPU nodes ceded to k8s (M2) — the anti-double-allocation control, shipped in Phase 1, not deferred.

Phase 2 (native supervision): `crates/spurd/src/cluster.rs` (owns the k0s unit), `spur-cluster` crate / `spurctld::cluster_k8s`, new `spur-proto` RPCs, `crates/spur-cli/src/k8s.rs` , `spur.amd.com/control-plane` role primitive, `discover_to_cdi()` on join.

Phase 3 (fidelity): `spur-device-plugin` binary + DaemonSet (depends on the standalone `spur-devices` lib), device-ID contract fix, RKE2/etcd HA.

7. On-hardware validation (over SSH)

```
# M0 – gates already verified 2026-07-09 (gfx950 / reachable / egress OK); re-run if hw changes
ssh gpu-node-1 'rocmfinfo | grep -oE "gfx[0-9a-f]+'" | sort -u' # → gfx950
ssh gpu-node-1 'ping -c2 <gpu-node-4-ip>' # → 0% loss to gpu-node-4 (underlay)
ssh gpu-node-1 'curl -sS -o /dev/null -w "%{http_code}\n" https://ghcr.io/v2/' # → 401 (reachable)
# after mesh init: sudo wg show spur0 ; ping -M do -s 1372 <peer-spur0-ip> (UDP/51820 + MTU)

# M1 – k0s + CNI/MTU
export KUBECONFIG=$PWD/spur-examples/gpu-k8s-arc-sglang/kubeconfig
kubectl get nodes -o wide # Ready, INTERNAL-IP=10.44.0.x
kubectl run a --image=busybox --restart=Never -- sleep 3600
kubectl run b --image=busybox --restart=Never -- sleep 3600 # different node
kubectl exec a -- sh -c 'ping -M do -s 1400 -c3 <b-ip>' # cross-node CNI + MTU

# M2 – GPU
kubectl get nodes -o json | jq '.items[].status.allocatable["amd.com/gpu"]'
kubectl apply -f rocmfinfo.yaml && kubectl logs rocmfinfo
spur nodes # GPU nodes DRAINED from scheduler

# M3/M4 – ARC + dind GPU
kubectl -n arc-systems get pods; kubectl -n arc-runners get pods
kubectl -n arc-runners exec <runner-pod> -c dind -- ls -l /dev/kfd # devices reach dind

# M5 – fork CI
gh workflow run fork-ci.yml -R powderluv/sglang
kubectl -n arc-runners get pods -w # EphemeralRunner on a GPU node

# M6/M7-ops – serving
kubectl -n sglang rollout status deploy/sglang
curl -s http://<ingress-or-nodeport>:30000/health
curl -s http://<ingress-or-nodeport>:30000/generate -d '{"text":"hello","sampling_params":{"max_new_tokens":16}}'

# M8 – native
spur k8s up && kubectl get nodes -o wide
sudo systemctl kill k0scontroller; sleep 20; kubectl get nodes # spur-owned unit auto-recovers
```

8. Risks & mitigations

- **Worker↔worker traffic silently broken (top risk).** Hub-and-spoke + no `ip_forward` passes kubelet registration but fails cross-node pod/east-west; and if the underlay has no worker↔worker path, full mesh is impossible. *Mitigation:* M0 underlay gate; full-mesh reconciliation; head `ip_forward` only for first light (funnels all GPU-collective bandwidth through the head).
- **Pod egress blackhole.** ARC long-poll + multi-GB pulls die if pod CIDR has no SNAT / workers lack egress. *Mitigation:* M0 egress gate + head MASQUERADE, verified before ARC.
- **MTU blackhole.** Large TLS/apiserver/ `kubectl exec` packets blackhole while ping succeeds. *Mitigation:* derive MTU from the underlay NIC, let flannel auto-derive, validate with a cross-node **pod-to-pod DF** transfer, not host ping.
- **GPU arch mismatch.** sglang images target gfx942/gfx950 only. *Mitigation:* M0 arch gate; else custom build or descope.
- **dind can't see the GPU.** `containerMode: dind` locks the sidecar. *Mitigation:* hand-authored pod spec with devices in the `dind` container (M4).
- **Double GPU allocation.** *Mitigation:* drain GPU nodes from spurctl in M2; per-runner `renderD` pinning; Phase-3 single authority.
- **k0s killed as a spur job.** The one-shot monitor + time-limit watchdog would drop/SIGKILL it. *Mitigation:* Phase-1 systemd; Phase-2 spur-owned unit, never the LaunchJob path.

- **Hardcoded LOCAL_DOCKER_REGISTRY inside mesh CIDR.** May route to a wrong mesh peer and hang. *Mitigation:* `--custom-image` bypass; exclude `10.44.14.109` from the AddressPool.
- **Privileged blast radius.** `dind` + GPU = broadly privileged pods; the repo-scoped App grants `Administration:write` and lives in a `k8s secret`. *Mitigation:* PodSecurity `privileged` only for `arc-runners`; single-repo App scope; sealed-secrets.
- **Cold-runner image re-pull / rate limits.** *Mitigation:* persistent docker data-root or node-local registry; DOCKERHUB creds.
- **Teardown finalizer hang.** *Mitigation:* uninstall runner sets before the controller (M7-ops).

- **RDMA container/driver ABI mismatch (from sglang's own DI CI).** If the serving/benchmark image's RDMA userspace (`rdma-core` + `ionic` provider) doesn't match the host `ionic_rdma` ABI (25.08.4.004), **MoRI reports "no active RDMA device"** and RDMA silently no-ops. *Mitigation:* assert `ibv_devinfo` lists `ionic_0..7` `PORT_ACTIVE` inside the container before trusting any DI run; build the image against the matching `ionic` userspace.
- **RDMA accidentally routed over spur0.** If NCCL/RCCL picks the WireGuard/pod interface instead of the RoCE NICs, bandwidth collapses. *Mitigation:* pin `NCCL_IB_HCA=ionic_*` + `NCCL_SOCKET_IFNAME=ens3` (rendezvous only); never let the data plane touch `spur0`.

9. Alternatives considered

Distro selection — k3s vs k0s vs RKE2 (verified against primary docs 2026-07-09):

- **k3s** bundles CoreDNS/Traefik/ServiceLB(Klipper)/local-path/metrics-server as `AddOn` CRs in `kube-system`, defaults to SQLite-via-**kine**, and runs the control plane *inside* the `k3s server` process (not pods). Several M6/M7-ops footguns (servicelb LB stuck pending, local-path RWO/naming, the NVIDIA `RuntimeClass`) are artifacts of that opinionated bundle. `flannel-iface spur0` is the simplest CNI-over-mesh, though.
- **k0s** ships **unmodified upstream** binaries, defaults to **real etcd**, keeps `kube-system` nearly empty (no vendor CRDs; MetalLB/NGINX/storage are opt-in Extensions), and **isolates the control plane** (no kubelet on controllers → matches `head=CPU` / `workers=GPU`). Decisively for this project, **k0sctl bootstraps multi-node over SSH from a declarative `k0sctl.yaml` inventory** (roles controller/worker) — exactly SPUR's inventory→fan-out model, and that file doubles as the reproducible cluster artifact. Costs: assemble ingress-nginx + MetalLB + local-path yourself; default CNI is **kube-router** so pick **calico-VXLAN** (built-in) or `custom` pinned to the mesh instead of k3s's one-flag flannel; device-plugin `hostPath` sits under `/var/lib/k0s/kubelet/device-plugins`; smaller AMD community.
- **RKE2** is the upstream-faithful middle path: control plane as **static pods in `kube-system`** (kubeadm-like), real etcd, CIS/FIPS hardening, Rancher+AMD GPU-operator docs; heavier than k0s but "looks like a normal upstream cluster."
- **Recommendation:** for the stated *pure-upstream* + *SPUR-owned-lifecycle* goals, prefer **k0s** (RKE2 as fallback if you want static-pod control-plane visibility / maximum AMD-Rancher support); keep **k3s** for throwaway/dev only. Everything above the bring-up+addon layer (ARC, sglang, SPUR supervision, the device-plugin concept, the mesh) is **distro-agnostic** — switching only reshapes M1 (bring-up→k0sctl), M2 (device-plugin path + CNI), and M6/M7-ops (install ingress/LB/storage explicitly).
- **fabric-provider-coexist (k3s as plain systemd, SPUR never owns it).** *Rejected as end-state* — delivers "K8s beside Spur," two independent resource managers per host with real GPU double-allocation and host contention; SPUR owns/accounts for zero pods. **Not discarded:** its systemd bring-up *is* Phase 0/1 here, because the target cluster is byte-identical to the embedded-k0s target; everything above the "who supervises k0s" line carries forward unchanged.
- **Reuse the existing `spur-k8s` operator to bootstrap.** *Rejected — category error.* The operator is a tenant needing the cluster to already exist (`Client::try_default()`, RBAC over an apiserver) that only creates Pods pinned via `spec.nodeName`; it cannot bootstrap or join a host itself. It can only be a downstream consumer after K8s-on-Spur exists.
- **kubeadm instead of a single-binary distro.** *Rejected for the first cut* — heavier (manual etcd/CNI/containerd) with more to wire over the mesh. k0s bundles unmodified upstream apiserver/scheduler/etcd/kube-proxy + CDI-capable containerd behind one binary and k0sctl. (RKE2 is the upstream-faithful fallback; see distro selection above.)
- **kind (as `k8s_test.sh` uses).** *Rejected* — Docker-in-Docker throwaway with no real GPU scheduling; fine for operator unit tests, useless for hosting GPU workloads.
- **ARC `containerMode: kubernetes`.** *Rejected* — forbids `docker build` / `docker run` (sglang CI shells out to docker), needs an RWO PVC per job, and doesn't wire GPUs into hook-spawned pods. `dind` is required.
- **Reimplement a k8s scheduler/apiserver in SPUR.** *Rejected* — the roadmap itself rejects "replace K8s scheduler entirely"; k0s ships the hard parts unmodified from upstream.

- **Relay overlay (Tailscale/Netbird) instead of WireGuard full mesh.** *Conditional fallback* — only if the M0 underlay gate shows workers can't reach each other directly; adds a dependency but provides DERP relays SPUR's mesh lacks.

10. Open questions for the user

1. **GPU part on the workers: RESOLVED — 8x MI355X (gfx950) per worker.** sglang half is viable via the `mi35x` image path.
2. **Worker $\Rightarrow$ worker underlay: RESOLVED — flat routed underlay, directly reachable.** Full mesh feasible; no relay overlay.
3. **Native depth this cycle:** stop at Phase 1 (k0sctl bring-up, ARC + sglang green) or push through Phase 2 (`spur k8s up` , `spurd-supervised`)? Phase 3 (native device plugin + HA) is a separate, larger effort.
4. **k8s distro: RESOLVED — k0s locked (2026-07-09).** Bring-up via k0sctl; calico-VXLAN pinned to the mesh; ingress-nginx + local-path installed explicitly; device-plugin under `/var/lib/k0s/kubelet` . Concrete tree: `spur-examples/gpu-k8s-arc-sglang/` . (RKE2 remains the fallback if static-pod control-plane visibility is later wanted.)
5. **GitHub auth:** GitHub App (recommended) vs classic PAT (`repo` scope, faster)?
6. **GPU partitioning:** dedicate whole GPU nodes to k8s (drain from `spurctl`), or split GPUs per manager via `ROCR_VISIBLE_DEVICES` ?
7. **External access scope:** is off-mesh/public reachability of the served model in scope (ingress + TLS + DNS), or is mesh-local NodePort enough?
8. **Serving scale + DI:** single-node serving, or multi-node TP / prefill-decode disaggregation? Multi-node needs the RoCEv2 backend fabric (M6.5, verified confirmed present) — confirm rail-IP assignment + PFC are provider-owned, and pick the DI path: **SPUR-Slurm** (`salloc` the 2 nodes, reusing sglang's DI CI) vs **k8s** pods. Also: ungated default model to avoid HF gating?

11. Reproducibility (everything-as-code)

Every change we make to the cluster is captured so a similar cluster is `git clone && ./bootstrap.sh` (and, later, `spur k8s up`). Tree: `spur-examples/gpu-k8s-arc-sglang/` (the `spur-examples` repo layout) — `README.md` (layout + intent), an **append-only RUNBOOK.md** (seeded with the 2026-07-09 read-only recon; every mutating command logged with host + result + rollback), `versions.lock.md` (distro/k8s/chart versions + image **digests**), and numbered dirs `00-network` $\rightarrow$ `40-serving` plus `bootstrap.sh` / `teardown.sh` . Target home: a dedicated `spur-examples` repo (e.g. `github.com/rocm/spur-examples`), as the `gpu-k8s-arc-sglang` example — kept **separate from the core rocm/spur code** so the runnable example evolves independently. Staged at `spur-examples/gpu-k8s-arc-sglang/` in this workspace, already in the `spur-examples` repo layout.

Discipline: declarative-first (a checked-in manifest/values/config over an ad-hoc command); pin images by digest; idempotent bring-up + reverse-ordered teardown (ARC runner sets before controller; k8s before mesh; re-admit drained SPUR nodes last); secrets via sealed-secrets/SOPS (only templates committed); anything done by hand is logged then promoted into a script; `make capture` snapshots live state as a drift check. **This tree is the concrete seed of the Phase-2 spur cluster capability** — with k0s, its `k0sctl.yaml` is the cluster definition SPUR would template and wrap.